

AstraFlow: Dataflow-Oriented Reinforcement Learning for Agentic LLMs

Haizhong Zheng¹, Yizhuo Di¹, Jiahui Wang¹, Shuwei Jin², Xueshen Liu², Yongji Wu³,

Z. Morley Mao², Ion Stoica³, Jiawei Zhao⁴, Beidi Chen¹

¹Carnegie Mellon University ²University of Michigan ³UC Berkeley ⁴Meta

Reinforcement learning (RL) is increasingly used to improve the reasoning, coding, and tool-use capabilities of large language models, but agentic RL remains prohibitively expensive. Scaling RL to agentic LLMs requires supporting complex workloads, including multi-policy collaborative training, while efficiently using elastic, heterogeneous, and cross-region compute resources. Existing LLM RL systems support some of these capabilities, but each new extension often requires dedicated system engineering. This burden arises from trainer-centered control architectures and the lack of principled abstractions for RL system components. To address these limitations, we propose AstraFlow, a dataflow-oriented RL system that replaces conventional trainer-centered control with principled component abstractions. In AstraFlow, rollout services, dataflow management, and training are decoupled into autonomous components, enabling the system to natively support complex multi-policy agentic RL workloads and efficiently exploit diverse compute resources. We evaluate AstraFlow across math, code, search, and AgentBench workloads, showing that the same system supports multi-policy training, elastic scaling, heterogeneous cross-region execution, and composable data algorithms without system-level code changes. In multi-policy collaborative training, AstraFlow achieves comparable or better accuracy than existing RL systems while speeding up training time by 2.7 $\times$.



Github: <https://github.com/Infini-AI-Lab/astraflow>

Website: <https://infini-ai-lab.github.io/astraflow>

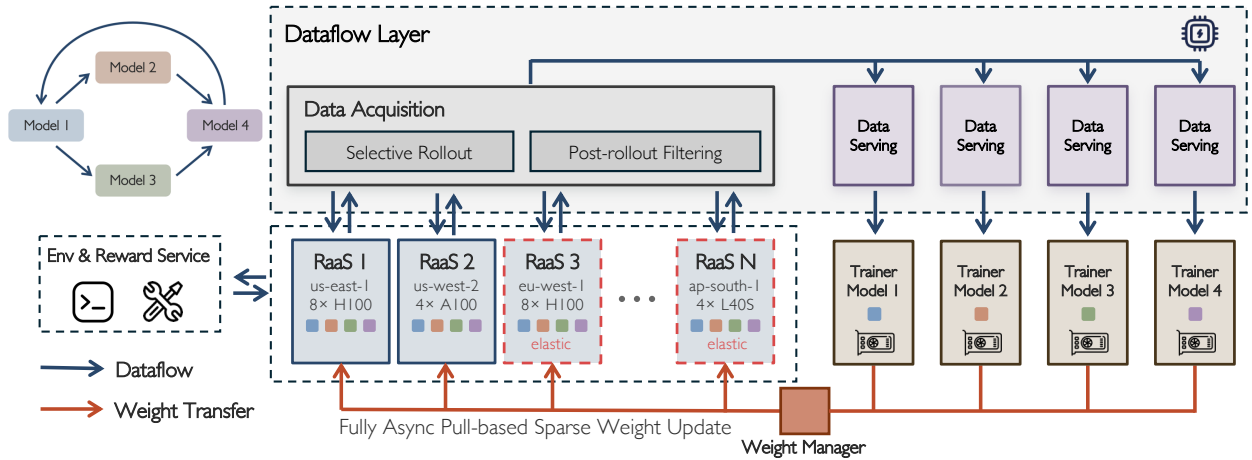


Figure 1 Overview of the AstraFlow architecture. A dataflow-oriented RL framework natively supports multi-policy collaborative training, elastic rollout, heterogeneous and cross-region rollout, and substitutable Rollout-as-a-Service (RaaS) and Trainer.

1 Introduction

Large language models (LLMs) are rapidly moving beyond standalone use into complex agentic systems, including coding agents [16, 34], search agents [6, 49], and multi-agent workflows [2, 17]. In these settings, reinforcement learning (RL) has emerged as a key technique for improving reasoning and tool-using capabilities [7, 25, 27, 31, 42].

Yet scaling RL for agentic systems remains challenging. It must accommodate the complexity of agentic workloads, including dynamic execution and multi-policy coordination, as well as the diversity of underlying compute environments, such as elastic and heterogeneous computing. This underscores the need for a general RL infrastructure that unifies agent execution, training, and resource management under a flexible and scalable system design.

Despite this need, existing systems [1, 12, 29, 36] remain constrained by rigid designs that limit their scalability and extensibility. **First**, existing LLM RL systems [29] are primarily designed for single-policy training. Their trainer-centered control logic coordinates rollout scheduling, data movement, policy optimization, and weight synchronization, making them rigid for multi-policy agentic RL, where collaborative training requires coordinating multiple policies and their interactions. **Second**, SOTA multi-agent systems [8, 35] primarily focus on serving. They are designed to execute complex agent workflows efficiently, but lack the training-time coordination needed to collaboratively optimize multiple policies. **Finally**, recent systems can be engineered to add individual capabilities such as multi-policy training [3, 45], elastic rollout [36], or heterogeneous rollout [11, 39]. Although these systems can support such capabilities, they do so through ad-hoc patches that require feature-specific system engineering on the existing design. Table 1 summarizes how existing systems support each capability but generally lack the abstractions needed to support and compose them natively.

Table 1 System-level comparison of LLM RL frameworks. ✓ denotes full support, ✗ no support, and *p* partial support.

Property	AstraFlow	AReaL	SLIME	verl [†]	RLBoost	Dr.MAS (verl)	prime-rl
Multi-policy collaborative training	✓	✗	✗	✗	✗	✓	✗
Substitutable trainer and rollout service	✓	✗	✗	✗	✗	✗	<i>p</i>
Modular data algorithm interfaces	✓	✗	✗	✗	✗	✗	✗
Fully asynchronous training	✓	✓	✓	✓	✗	✗	✓
Disaggregated rollout-training architecture	✓	✓	✓	✓	✓	✗	✓
Runtime elastic rollout scaling	✓	✗	✗	✗	✓	✗	<i>p</i>
Cross-region / heterogeneous rollout	✓	✗	✗	✗	✗	✗	<i>p</i>

[†]The original verl paper does not present fully asynchronous or disaggregated architecture; these entries reflect support later added in the open-source verl repository.

Building an ideal argentic RL system for LLMs requires rethinking several assumptions built into conventional LLM RL systems. First, such a system should treat multiple trainable policies and trainers as first-class components, rather than assuming a single-model, single-trainer control workflow. Second, it should also move beyond the assumption of a fixed compute environment, allowing workloads to seamlessly run across heterogeneous, cross-region, or elastic resources through clean execution interfaces. Third, it should move beyond tightly coupled implementations by exposing simple interfaces between rollout engines, trainers, and data algorithms, so that each component can be replaced or extended independently.

Our key insight is that the limitation of existing RL systems comes from a single trainer-centered control loop and the lack of principled abstractions among RL components. To address this, we propose **AstraFlow**, a dataflow-oriented RL training system for agentic LLMs. As shown in Fig. 1, AstraFlow consists of three components: a dataflow layer, Rollout-as-a-Service (RaaS), and trainers. **1) Dataflow layer.** The dataflow layer coordinates rollout, training, and data-processing components through shared data, rather than centralized trainer control (Section 3.2). This enables autonomous rollout services and trainers to compose naturally, supports multi-policy collaborative training, and expresses policies such as curriculum scheduling, replay, data mixing, filtering, sampling, and staleness correction as dataflow policies. **2) Rollout-as-a-Service.** RaaS decouples trajectory generation from policy optimization through rollout interfaces (Section 3.3). This allows users to plug in optimized agent inference engines or specialized rollout backends without modifying trainers or system orchestration. It also enables rollout components to scale independently across heterogeneous, cross-region, and elastic compute resources. **3) Trainers.** Trainers consume data from the dataflow layer and publish updated weights back to the system (Section 3.4). Since they no longer directly control rollout scheduling, data movement, or rollout-runtime details, trainers become independently replaceable. This makes it easy to integrate fault-tolerant trainers, specialized optimizers, or multiple trainers for multi-policy learning without changing the rest of the system.

In the evaluation part, we demonstrate the flexibility of AstraFlow from three perspectives: multi-policy collaborative RL, system flexibility, and data algorithm flexibility. For multi-policy collaborative RL, we evaluate AstraFlow on three multi-policy workflows, achieving comparable or better accuracy than the existing multi-agent RL system while delivering an up to 2.7× speedup in training. Also, to the best of our knowledge, even without any multi-agent-specific system modifications, AstraFlow is the first fully asynchronous

multi-policy collaborative RL framework. For system flexibility, we first show that, without requiring any code changes, rollout auto-scaling can be achieved with an agentic maintainer. Then we show that AstraFlow natively supports heterogeneous and cross-region training without feature-specific engineering. For data flexibility, we demonstrate the flexibility of the dataflow-layer abstraction by integrating and composing data algorithms, including dynamic sampling [42], GRESO [47], and buffer replay. Together, we demonstrate that, thanks to the dataflow-oriented RL design, AstraFlow natively supports multi-policy collaborative training, diverse compute environments, and composable data algorithms without feature-specific system code.

2 Related Work

2.1 RL for Agentic LLMs

RL has become a central post-training technique for improving LLM reasoning, code generation, and tool-use capabilities [7, 25, 27, 31, 42]. Many efforts improve the performance, stability, and efficiency of RL itself, including better policy-optimization objectives [26, 43, 46], reward design [33], off-policy or asynchronous training [24, 48], and data-centric algorithms [30, 37, 38, 47] that decide which prompts for sampling, which trajectories to keep, and which batches to train on. At the same time, RL workloads are expanding from single-turn reasoning to more complex agentic settings [1, 32, 44] such as software-engineering agents [16, 34], search tasks [49], and os environment interaction workflows [18]. These workloads introduce heterogeneous rollouts with variable lengths, tool feedback, intermediate artifacts, and data-policy interventions throughout training. Recent multi-agent RL workloads [3, 45] require collaboration among multiple trainable policies, further complicating training orchestration. Together, these trends make LLM RL workloads more complex and expensive, creating a need for better system support to run them efficiently on hardware resources.

2.2 LLM RL Training Frameworks

A typical RL training pipeline for LLMs has two major stages: 1) *rollout*, where inference engines generate trajectories and rewards from the current policy, and 2) *training*, where a trainer consumes rollout data and updates the policy. Existing LLM RL systems [1, 9, 10, 12, 12–14, 21, 28, 29, 36, 50, 51] mainly organize these two stages in two ways. **Colocated synchronous systems** such as verl [29], Real [22], and RLHFuse [51] place training and rollout on the same GPU pool and alternate between trajectory generation and optimization. This design guarantees the on-policy training, but it suffers from long-tail rollout latency, leaving expensive trainer GPUs idle during rollout. **Disaggregated RL systems** such as AReaL [5] and SLIME [52] address this utilization problem by decoupling rollout generation from policy optimization, allowing rollout workers and trainers to run on separate GPU pools and overlap execution. However, the heterogeneity between rollout and training in LLM RL complicates scheduling, resource allocation, and synchronization, while enabling optimizations such as elastic scaling [36] and heterogeneous resource management [11, 39].

3 Dataflow-Oriented RL for Agentic LLMs

In this section, we present the design of AstraFlow. We begin by motivating the shift from trainer-centered control to dataflow-oriented coordination, explaining why compute decoupling alone is insufficient for agentic RL in Section 3.1. We then introduce the three abstraction designs, as shown in Figure 1: a *dataflow layer* that manages rollouts and training batches (Section 3.2); a *Rollout-as-a-Service* abstraction that decouples trajectory generation from optimization (Section 3.3); and a *trainer* abstraction that consumes batches, updates policies, and publishes weights (Section 3.4).

3.1 Motivation: From Trainer-Centered Control to Dataflow-Oriented Coordination

Compute decoupling is not enough. Although disaggregated RL frameworks [5, 52] separate rollout and training computation, this separation is primarily a compute-placement mechanism, not a principled component abstraction. Rollout scheduling, data selection, replay, staleness handling, and weight synchronization often remain embedded in a trainer-centered control loop. As a result, new capabilities tend to require feature-specific system changes. Multi-policy collaborative training, for instance, requires coordinating multiple independently

trained policies, trainers, and weight streams. Elastic, heterogeneous, or cross-region rollouts require additional mechanisms for workers to join and leave dynamically and for weights to be transferred under resource constraints. These capabilities can be added to existing systems, but usually through ad hoc patches or substantial redesign, like Areal-Hex [39]; composing several of them only amplifies the complexity. **The root cause behind this limitation is the lack of clean abstraction boundaries among rollout execution, dataflow management, training, and weight transfer.** Without these boundaries, new capabilities cannot be supported naturally by the system design and instead require explicit feature-specific engineering. Table 1 summarizes this abstraction gap: existing LLM RL frameworks may support asynchrony or rollout-training disaggregation, but generally lack abstractions for composing them.

Dataflow-oriented coordination. Motivated by this abstraction gap, we propose dataflow-oriented RL for agentic LLMs, a design principle implemented in AstraFlow. The key insight is that disaggregation should not only separate rollout and training computation, but also separate their control responsibilities. Instead of organizing the system around a trainer-centered control loop, AstraFlow uses dataflow-oriented coordination: rollout services, trainers, and the dataflow layer each run *autonomous control loops* and interact only through minimal data and weight interfaces. These interfaces turn rollout, training, data management, and weight transfer into composable system boundaries. As a result, capabilities such as multi-policy collaborative training, elastic rollout pools, heterogeneous and cross-region rollouts, and modular data algorithms can be expressed by the system architecture itself, rather than added through feature-specific engineering.

Design challenges. To realize this design, AstraFlow must address three challenges. First, it must provide a data coordination layer that manages prompts, trajectories, rewards, batching, routing, replay, and staleness across multiple rollout services and trainers without returning control to a single trainer loop. Second, it must expose stable component boundaries so that rollout engines, trainer backends, and data algorithms can be replaced or extended without pipeline rewrites. Third, it must support asynchronous and bandwidth-efficient weight flow across multiple policies and rollout pools, including elastic, heterogeneous, and cross-region deployments.

3.2 Dataflow Layer Abstraction

The dataflow layer is the coordination plane between rollout services and trainers. The layer represents RL data in its natural units, including prompts, trajectories, metadata, and training batches. RaaS nodes pull rollout tasks from the layer and push completed trajectories back, while trainers independently pull batches according to their own optimization loops.

Data algorithm interface. The dataflow layer exposes a programmable interface for algorithms that operate on prompts, trajectories, rewards, and metadata. Policies such as selective rollout, curriculum scheduling, post-rollout filtering, dynamic sampling [42], replay, data mixing, and staleness correction can therefore be implemented as dataflow policies without modifying the trainer, RaaS implementation, or system orchestration.

Data-driven coordination. The dataflow layer also coordinates autonomous rollout services and trainers through data availability and routing. Although each component runs its own control loop, the layer can regulate their interaction by deciding which rollout tasks, trajectories, and batches each component receives. For example, it can throttle slow or stale rollout services by assigning fewer tasks, prioritize fresher trajectories for a trainer, or block unsuitable batches through backpressure. In multi-policy training, trajectory metadata such as producing policy, model version, timestamp, reward statistics, and task type allows the layer to route policy-specific, shared, or mixed data streams to different trainers without requiring direct trainer-to-trainer coordination.

Together, these two roles make the dataflow layer both a modular data-algorithm interface and a control plane for independent components. New data policies and coordination strategies can be added in the dataflow layer

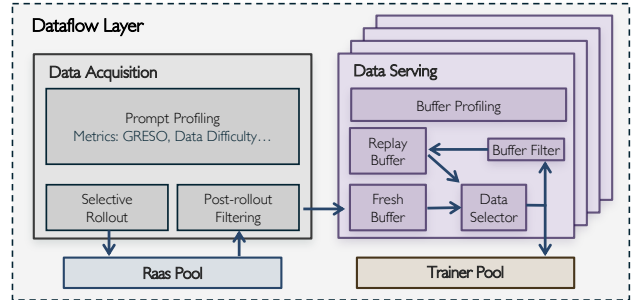


Figure 2 Dataflow layer abstraction. Prompt sources, RaaS nodes, and trainers interact through a shared layer that buffers data and applies sampling, filtering, and routing policies.

rather than by rewriting a trainer-centered control loop.

3.3 Rollout-as-a-Service (RaaS) Abstraction

RaaS models rollout generation as a pure *agent-serving service*. Each RaaS node receives tasks from the dataflow layer, executes the corresponding agent workflow, and returns trajectories. The RaaS interface only requires that the service consume tasks, produce trajectories, and refresh weights through the trainer-side weight-transfer interface. This interface makes rollout execution substitutable. An efficient agent-serving runtime can be plugged into AstraFlow as long as it follows the RaaS contract. The runtime does not need to know how trajectories are sampled, replayed, filtered, or assigned to trainers. Likewise, the trainer does not need to know which serving runtime produced the trajectory. This separation allows AstraFlow to reuse specialized agent-serving systems as rollout backends instead of re-implementing their internal execution logic.

RaaS also makes rollout capacity elastic. Adding capacity means launching more RaaS nodes that connect to the same dataflow layer and weight-transfer interface. Removing capacity, slow workers, or failures affect only the rate at which trajectories arrive, not the independent trainer control loop. This property is especially useful for heterogeneous and cross-region settings, where rollout services may have different latency, throughput, and network bandwidth.

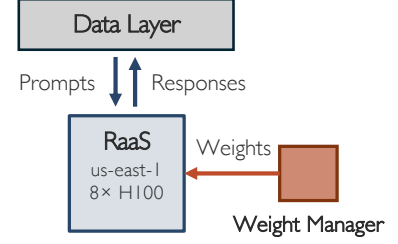


Figure 3 RaaS abstraction. Each rollout node consumes tasks, produces trajectories, and refreshes weights.

3.4 Trainer Abstraction and Weight Transfer

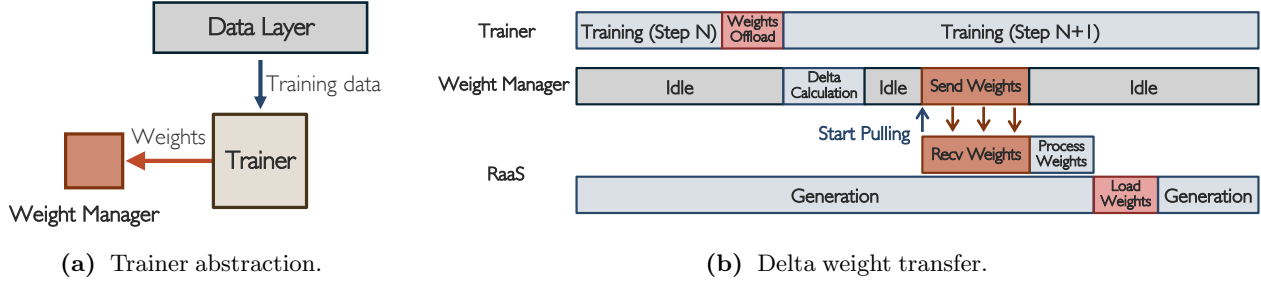


Figure 4 (a) Each trainer consumes batches from the dataflow layer and publishes updated weights through a trainer-side weight interface. (b) Fully async pull-based sparse weight update. A trainer consumes batches from the dataflow layer, performs optimization with its own backend, and publishes updated weights through a trainer-side weight-transfer interface. From the trainer’s perspective, the dataflow layer behaves like a streaming training corpus, and the weight-transfer interface behaves like a publication target. The trainer does not need to manage rollout workers, serve model weights directly, or coordinate with other trainers.

This abstraction also makes training backends substitutable. An existing RL, SFT, or fault-tolerant training backend can participate in AstraFlow if it can consume batches from the dataflow layer and publish weights through the same trainer abstraction. The same interface also supports multi-policy collaborative training: each policy can have an independent trainer and weight stream, while the dataflow layer controls how trajectories are distributed to each trainer.

Weight Transfer. Within the trainer abstraction, the weight-transfer mechanism owns the weight flow between trainers and rollout services. It stores model versions, exposes the latest or requested versions to RaaS nodes, and handles asynchronous refresh. Because RaaS nodes pull weights when appropriate, weight delivery is not part of the trainer’s critical path. The trainer-side weight interface can implement full-model transfer, sparse transfer, and version-aware refresh behind the same abstraction. This design keeps constrained or remote weight transfer isolated from trainer logic while allowing rollout services to refresh at different rates.

4 Evaluation: Applications of AstraFlow

In this section, we demonstrate the flexibility of AstraFlow from three perspectives: multi-policy collaborative training, system flexibility, and data algorithm flexibility.

- Section 4.1 evaluates AstraFlow on three two-policy workflows, demonstrating improvements over matched single-policy baselines and a $2.7\times$ speedup in training time.
- Section 4.2.1 demonstrates that rollout auto-scaling can be achieved using an agentic maintainer, without requiring any code changes.
- Section 4.2.2 highlights AstraFlow’s native support for heterogeneous and cross-region training without feature-specific engineering.
- Section 4.3 validates the dataflow-layer abstraction by integrating and composing data algorithms, including dynamic sampling [42], GRESO [47], and buffer replay.

Due to the space limitation, we include our detailed experimental setting in Appendix A.

4.1 Application I: Multi-Policy Collaborative Training

We first evaluate AstraFlow’s flexibility on multi-policy collaborative RL through three multi-agent workflows illustrated in Figure 5: math solver-verifier, code solver-selector, and code solver-test-case generation. In AstraFlow, users only specify the multi-agent workflow: role order, context passing, and reward assignment. Existing multi-agent RL systems such as Dr. MAS [3] and Stronger MAS [45] build this support on top of verl [29], requiring pipeline-level modifications to coordinate multiple roles, policies, trainers, and weight streams, while also inheriting verl’s colocated synchronous execution. In contrast, AstraFlow natively supports multi-policy workflows through its dataflow-oriented RL abstraction. As a result, multi-agent RL becomes a workflow-level change rather than a system-level pipeline modification. To the best of our knowledge, AstraFlow is the first LLM RL framework to support fully asynchronous multi-policy collaborative training.

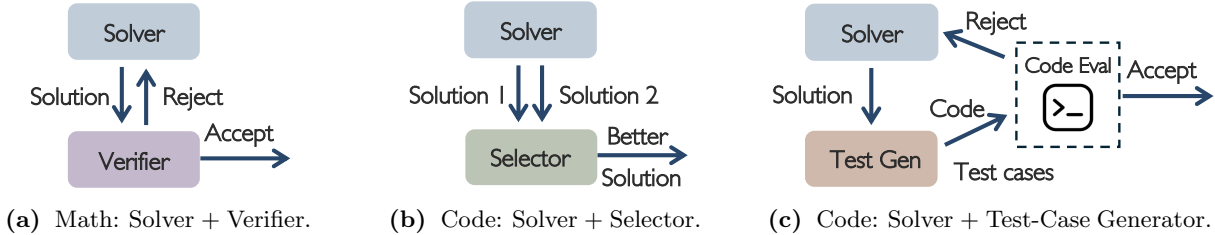


Figure 5 Multi-agent workflows evaluated: (a) Math solver + verifier, (b) Code solver + selector, and (c) Code solver + test-case generator with retry. Different policies serve different roles in a workflow.

Multi-agent Workflow. For every multi-policy run, both roles are initialized from Qwen3-8B [41] and trained as separate policies. In the math workflow, a Solver proposes a solution and a Verifier accepts or rejects it; if rejected, the Solver receives feedback and retries. In the code-selector workflow, the Solver generates two candidate programs and the Selector chooses one to submit. In the code test-case workflow, the Solver writes a program, the Test-Case Generator produces tests, an evaluator executes the program on those tests, and the Solver retries on failure. Full prompt templates for each role are given in Appendix A.9.

Table 2 Math multi-policy training results under matched conditions. Solver and Solver + Verifier (verl) accuracy follows Dr. MAS. AstraFlow achieves comparable or better accuracy while reducing iteration time by $2.7\times$.

Method	AIME24	AIME25	MATH500	Minerva	Avg Acc.	Time/iter (s)
Solver	42.9	31.8	90.5	39.2	51.1	—
Solver + Verifier (verl)	44.6 (+1.7)	41.5 (+9.7)	90.7 (+0.2)	40.9 (+1.7)	54.4 (+3.3)	212.64
Solver + Verifier (AstraFlow)	47.3 (+4.4)	40.6 (+8.8)	92.9 (+2.4)	45.0 (+5.8)	56.5 (+5.4)	77.65

$2.7\times$ speeds up on multi-agent math training. Table 2 reports the math multi-agent training results. Both solver-verifier configurations outperform the single-policy Solver baseline, indicating that collaborative multi-

policy training provides an effective learning signal. In particular, AstraFlow improves average accuracy from 51.1% to 56.5%, a gain of 5.4%, which is comparable to or better than the verl-based Dr. MAS implementation. At the same time, AstraFlow is substantially more training-efficient than Dr. MAS: because Dr. MAS inherits verl’s colocated synchronous execution, long-tail multi-agent rollouts can stall the entire iteration. By contrast, AstraFlow reduces the time per iteration from 212.64s to 77.65s.

Generality to code multi-agent workflows. Table 3 evaluates whether the same multi-policy abstraction extends beyond math to code generation. We consider two interaction patterns: Solver + Selector, where the Solver generates two candidates and the Selector chooses one to submit, and Solver + Test-Case Generator, where generated tests provide execution feedback for retry. Both workflows improve over the matched single-policy Solver on every benchmark, with the stronger workflow raising average accuracy from 30.29% to 34.55% (+4.26). We use the matched Solver baseline rather than a specialized code-agent baseline because this experiment is intended to test system generality under controlled training conditions; Table 2 provides the direct system-to-system efficiency comparison. The result shows that AstraFlow can express substantially different multi-agent code workflows by changing only workflow logic, while reusing the same trainer, rollout, dataflow, and weight interfaces. Figure 6 plots the training-time eval accuracy averaged over the three code benchmarks.

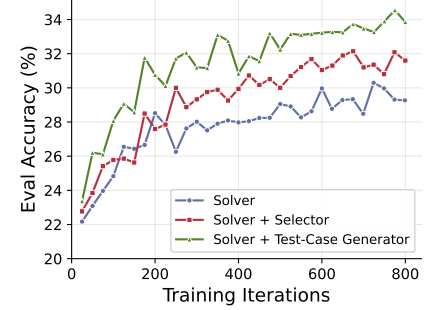


Figure 6 Eval accuracy during training (Qwen3-8B), averaged over LiveCodeBench v5/v6 and Codeforces.

Table 3 Code-generation accuracy of single-policy vs. multi-policy collaborative training with Qwen3-8B [41]. We report two multi-policy workflows.

Method	LiveCodeBench v5	LiveCodeBench v6	Codeforces	Avg
Solver	36.83	32.86	21.20	30.29
Solver + Selector	38.32 (+1.49)	35.43 (+2.57)	22.67 (+1.47)	32.14 (+1.85)
Solver + Test-Case Generator	41.62 (+4.79)	36.29 (+3.43)	25.74 (+4.54)	34.55 (+4.26)

Takeaway. Across math and code, AstraFlow supports multiple two-policy workflows through the same dataflow-oriented abstraction, improving over matched single-policy baselines while achieving a $2.7\times$ iteration-time speedup in the direct math comparison.

4.2 Application II: System Flexibility

We next evaluate AstraFlow’s system flexibility through two case studies: (i) zero-code rollout auto-scaling driven by an agentic maintainer (Section 4.2.1), and (ii) heterogeneous and cross-region training over throttled GPUs and limited-bandwidth links (Section 4.2.2). For both case studies, no feature-specific engineering is required: the same RaaS abstraction (Section 3.3) and weight-transfer runtime (Section 3.4) absorb auto-scaling, GPU heterogeneity, and slow remote synchronization without any change to the AstraFlow system.

4.2.1 Case Study I: Rollout Auto-Scaling with an Agentic Maintainer

Although AstraFlow does not control the trainer or the rollout services directly, the dataflow layer sits on the path that connects them and can therefore observe the workload balance: how many trajectories each rollout pool produces, how many the trainer consumes, and how long the trainer spends waiting on data. Every K trainer steps, AstraFlow exports a summary of these three quantities, the trainer waiting fraction w , the recent production and consumption counts n_p and n_c , and the availability of each rollout service, and uses them to derive a target rollout-pool size with a simple three-zone policy, applying a dead band $[\tau_{\text{low}}, \tau_{\text{high}}]$ on w :

$$G_{\text{target}} = \begin{cases} \lceil G/(1-w) \rceil & \text{if } w > \tau_{\text{high}}, \\ \min(G, \lceil G \cdot (n_c/n_p) \cdot \rho \rceil) & \text{if } w < \tau_{\text{low}}, n_p, n_c > 0, \\ G & \text{otherwise,} \end{cases} \quad (1)$$

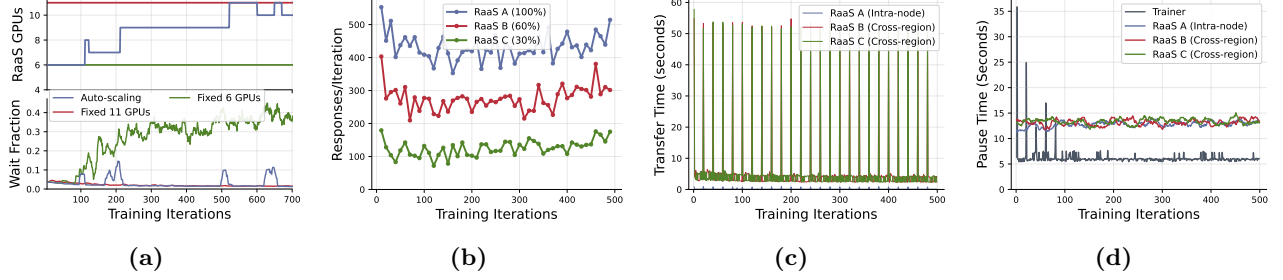


Figure 7 System-flexibility experiments. (a) Rollout GPU count and trainer waiting time under different rollout-pool settings. (b) Per-iteration rollout throughput from different RaaS pools. (c) Weight-transfer time across local and remote rollout pools. (d) Trainer and rollout downtime in the heterogeneous cross-region setting.

where G is the current rollout GPU count and ρ is the shrink-margin factor. The policy adds GPUs when the trainer is starving ($w > \tau_{\text{high}}$), removes GPUs when the pool is over-allocated ($w < \tau_{\text{low}}$).

Table 4 Accuracy, training time, and GPU-hour comparison across the three rollout-pool configurations on the Qwen3-14B [41] math job. Auto-scaling keeps wall-clock time close to the over-allocated baseline while reducing total GPU-hours. Bold marks the best GPU-hour costs.

RaaS Strategy	Avg Acc.	Wall (h)	Rollout GPU-h	Trainer GPU-h	Total GPU-h	Wait frac.
Fixed 6 GPUs	68.6	35.8	214.8	143.2	358.0	26.9%
Fixed 11 GPUs	68.0	23.9	263.4	95.8	359.2	2.1%
Auto-scaling	67.9	24.4	214.5	97.4	312.0	3.0%

No-code auto-scaling via an agentic maintainer. The RaaS abstraction (Section 3.3) makes each rollout service hot-swappable: the dataflow layer treats instances as interchangeable producers and tolerates them joining or leaving at runtime, so resizing the rollout pool reduces to launching or retiring RaaS instances on demand. On top of this, the utilization report above already names a target pool size G_{target} , leaving the maintainer with a purely operational job: read the report and act on the suggested target. We use Claude Code as the agentic maintainer in our experiment: it runs alongside the training job, polls the utilization summary every K steps, follows the suggested G_{target} , and issues the corresponding cluster commands to launch or retire RaaS instances—preferentially removing unavailable or persistently low-throughput services on scale-down. AstraFlow contains no scheduler-specific code; porting the loop to Slurm, Kubernetes, or an in-house scheduler only requires adapting the maintainer’s instructions to the corresponding cluster interface.

Auto-scaling minimizes GPU-hours with comparable accuracy. We compare auto-scaling against two fixed rollout-pool baselines that match the lower and upper ends of the controller’s scaling range: 6 rollout GPUs and 11 rollout GPUs. Table 4 reports the resulting accuracy, wall-clock time, trainer wait fraction, and GPU-hour cost. The 6-GPU baseline uses few rollout resources but starves the trainer, while the 11-GPU baseline removes most trainer waiting by keeping the larger pool allocated for the full run. Auto-scaling achieves comparable accuracy and nearly the same wall-clock time as the 11-GPU setting, with a small increase from 23.9h to 24.4h. However, by releasing rollout capacity when it is not needed, it reduces total GPU-hours to 312.0, about 13% lower than both fixed-pool baselines. Figure 7a traces the loop in action: the maintainer scales up when trainer waiting rises, holds inside the dead band, and scales down when sustained low waiting indicates over-allocation.

4.2.2 Case Study II: Heterogeneous and Simulated Cross-Region Training

We simulate a cross-region heterogeneous deployment using three nodes (a four-GPU trainer plus three four-GPU RaaS pools, one local and two remote) and train Qwen3-14B [41] with M2PO on math. GPU heterogeneity is induced by per-GPU power caps—700 W on the local pool and 400 W and 250 W on the two remote pools—yielding rollout-throughput shares of roughly 100%, 60%, and 30% respectively in Figure 7b. To emulate the cross-region paths, the two remote links are further shaped to 4 Gbit/s effective bandwidth and 300 ms round-trip latency. All three pools contribute to every iteration and the trainer never blocks on any single pool.

Training quality is preserved despite the constrained network: the cross-region run reaches 67.6 average accuracy on the five-benchmark math suite, comparable to a homogeneous local baseline. Figure 7c reports per-iteration weight-transfer time on each link. With $\geq 98.9\%$ delta sparsity for Qwen3-14B [41] (Figure 8), the per-iteration payload drops from a ~ 28 GB full sync to roughly 1.5 GB of delta bytes, so most remote transfers complete in tens of seconds even at 4 Gbit/s and 300 ms RTT. The periodic full syncs (every 20 iterations) appear as the visible spikes; they remain bounded because they are amortized over many delta steps.

Figure 7d shows that even the slow full-sync transfers do not block the training loop. On the rollout side, downtime is essentially constant across the run: just the SGLang reload and prefill window after each weight update, independent of how the bytes arrived. On the trainer side, the early iterations show occasional waiting because the previous iteration’s full transfer is still finishing on a remote pool; as training progresses and generation lengths grow, the training phase becomes long enough to fully overlap the next iteration’s transfer, and trainer downtime converges to a near-constant baseline. This is the behavior the request-based delta-pull design is intended to deliver (Section 3.4): the cost of a slow link is masked by ongoing training work, so heterogeneous rollout and slow remote synchronization are absorbed by the existing abstractions without any change to the trainer, RaaS, or dataflow interfaces.

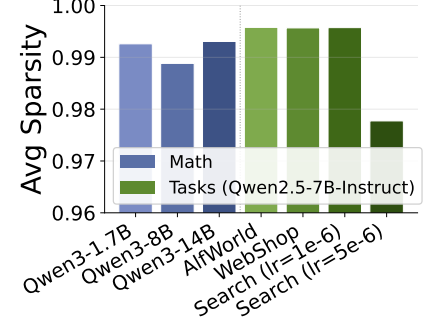


Figure 8 Weight delta sparsity on different models and tasks.

Remote Sparse Weight Transfer. As discussed in recent works [4, 23], per-iteration weight updates in RL training are extremely sparse: under bf16 most parameters are bit-exactly identical from one iteration to the next, so the trainer needs to ship only a tiny fraction of its weights to the rollout fleet. Figure 8 measures this directly across two axes: model scale (Qwen3-1.7B / 8B / 14B [41] on math) and task (Qwen2.5-7B on AlfWorld, WebShop, and Search), reporting the fraction of bf16 parameters bit-exactly equal to the previous iteration, averaged over the first 500 training iterations. At a fixed learning rate, sparsity is largely independent of model size and task: every math run lands in 0.989–0.993 and the Qwen2.5-7B tasks all reach ≥ 0.996 at $lr = 1e-6$. Learning rate is the dominant driver: raising it to $5e-6$ on Search drops sparsity to 0.978, since larger updates push more parameters past the bf16 ULP boundary, but even in this most aggressive regime sparsity still exceeds 0.97, leaving a $\geq 30\times$ compression upper bound on the bytes the trainer must ship per iteration. The request-based remote weight-transfer path can therefore rely on a high-sparsity assumption across all workloads we measured; per-iteration sparsity curves are deferred to Appendix B.2.

4.2.3 Performance Comparison to Existing RL Framework

Table 5 Performance comparison against AReaL on Qwen3-1.7B and Qwen3-8B [41] M2PO math reasoning tasks training for 800 iterations. AstraFlow matches AReaL’s accuracy and per-iteration efficiency at both scales while providing all the flexibility shown in the previous sections at no cost.

Model	Framework	AIME24	AIME25	AMC	MATH500	Minerva	Avg	Time/iter (s)	Time/1M tok (s)
Qwen3-1.7B	AReaL	32.5	27.5	61.1	88.2	38.2	49.5	81.5	70.1
	AstraFlow	33.3	30.6	59.2	87.5	35.8	49.3	81.1	69.4
Qwen3-8B	AReaL	60.0	55.0	72.2	94.6	45.2	65.4	137.0	117.6
	AstraFlow	62.3	50.0	72.5	94.9	44.5	64.8	139.6	119.6

We train Qwen3-1.7B and Qwen3-8B [41] math jobs with M2PO on AstraFlow and on AReaL [5], a representative trainer-centric RL framework, and compare accuracy and training speed (Table 5). At both scales, the two systems reach comparable accuracy on the math suite (within 0.2 / 0.6 points on average) and comparable per-iteration training speed (within 1% / 2%): AstraFlow matches AReaL’s performance and efficiency on a workload AReaL is specialized for, while still providing the flexibility demonstrated in the previous sections.

4.3 Application III: Data Algorithm Flexibility

We train Qwen3-8B [41] on math with three representative data algorithms, chosen to cover the three intervention points along the RL data path: *GRESO* [47] performs **selective rollout**, deciding which prompts are sent to

generation; *dynamic sampling* [42] performs **post-rollout filtering**, discarding zero-advantage trajectories after generation; and *buffer replay* performs **training-batch selection**, reusing useful trajectories across multiple trainer updates. Together they exercise pre-, post-, and serving-side hooks of the dataflow layer.

Figure 9 reports accuracy versus generated rollouts for the three algorithms. Dynamic sampling [42] lifts final accuracy meaningfully but increases generation cost by roughly $3.5\times$ (about 200k $\rightarrow$ 700k rollouts), since post-filtering discards a large fraction of generations. GRESO [47] and buffer replay sit on the opposite side of the trade-off: both reach baseline-level accuracy with substantially fewer generated rollouts—GRESO [47] by avoiding low-value prompts before rollout, buffer replay by reusing trajectories at batch serving. This data algorithm composition shows that: each algorithm is a self-contained policy class that the dataflow layer imports as a plug-in, so prompt selection, rollout filtering, and trajectory replay become composable, modular components rather than system-wide rewrites.

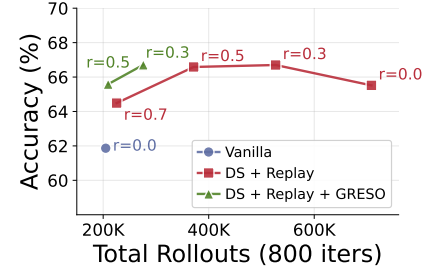


Figure 9 Math accuracy versus generated rollouts for dataflow-layer data algorithms.

5 Conclusion

We presented AstraFlow, a dataflow-oriented RL training system for agentic LLMs. Our central observation is that the rigidity of existing RL training systems comes from a single architectural choice: the trainer sits at the center of the loop and orchestrates everything else. AstraFlow replaces this trainer-centered control with three narrow abstractions, a dataflow layer for coordination, a rollout service for trajectory generation, and a trainer for policy updates, that interact only through stable interfaces. Because no component owns the global control flow, properties such as full asynchrony, elastic rollout scaling, plug-in trainers and inference engines, multi-trainer multi-policy coordination, and efficient delta and request-based sparse weight transfer all fall out of the same design rather than being bolted on as separate mechanisms.

References

- [1] Shiyi Cao, Dacheng Li, Fangzhou Zhao, Shuo Yuan, Sumanth R Hegde, Connor Chen, Charlie Ruan, Tyler Griggs, Shu Liu, Eric Tang, et al. Skyrl-agent: Efficient rl training for multi-turn llm agent. *arXiv preprint arXiv:2511.16108*, 2025.
- [2] Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, et al. Why do multi-agent llm systems fail? *arXiv preprint arXiv:2503.13657*, 2025.
- [3] Lang Feng, Longtao Zheng, Shuo He, Fuxiang Zhang, and Bo An. Dr. mas: Stable reinforcement learning for multi-agent llm systems. *arXiv preprint arXiv:2602.08847*, 2026.
- [4] Fireworks AI. Frontier rl is cheaper than you think. <https://fireworks.ai/blog/frontier-rl-is-cheaper-than-you-think>, March 2026. Fireworks.ai Blog.
- [5] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. Areal: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.
- [6] Jiaxuan Gao, Wei Fu, Mingyang Xie, Shusheng Xu, Chuyi He, Zhiyu Mei, Banghua Zhu, and Yi Wu. Beyond ten turns: Unlocking long-horizon agentic search with large-scale asynchronous rl. *arXiv preprint arXiv:2508.07976*, 2025.
- [7] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [8] Shanshan Han, Qifan Zhang, Weizhao Jin, and Zhaozhao Xu. Llm multi-agent systems: Challenges and open problems. *arXiv preprint arXiv:2402.03578*, 2024.
- [9] Zhenyu Han, Ansheng You, Haibo Wang, Kui Luo, Guang Yang, Wenqi Shi, Menglong Chen, Sicheng Zhang, Zeshun

- Lan, Chunshi Deng, et al. Asyncflow: An asynchronous streaming rl framework for efficient llm post-training. *arXiv preprint arXiv:2507.01663*, 2025.
- [10] Jingkai He, Tianjian Li, Erhu Feng, Dong Du, Qian Liu, Tao Liu, Yubin Xia, and Haibo Chen. History rhymes: Accelerating llm reinforcement learning with rhymerrl. *arXiv preprint arXiv:2508.18588*, 2025.
 - [11] Yongjun He, Shuai Zhang, Jiading Gai, Xiyuan Zhang, Boran Han, Bernie Wang, Huzefa Rangwala, and George Karypis. Hetrl: Efficient reinforcement learning for llms in heterogeneous environments. *arXiv preprint arXiv:2512.12476*, 2025.
 - [12] Brad Hilton, Kyle Corbitt, David Corbitt, Saumya Gandhi, Angky William, Bohdan Kovalevskyi, and Andie Jones. Art: Agent reinforcement trainer. <https://github.com/openpipe/art>, 2025.
 - [13] Jian Hu, Xibin Wu, Wei Shen, Jason Klein Liu, Zilin Zhu, Weixun Wang, Songlin Jiang, Haoran Wang, Hao Chen, Bin Chen, et al. Openrlhf: An easy-to-use, scalable and high-performance rlhf framework. *arXiv preprint arXiv:2405.11143*, 6, 2024.
 - [14] Prime Intellect. Prime-rl, 2025. URL <https://github.com/PrimeIntellect-ai/prime-rl>.
 - [15] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
 - [16] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.
 - [17] Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-rl: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025.
 - [18] Hanyu Lai, Xiao Liu, Yanxiao Zhao, Han Xu, Hanchen Zhang, Bohao Jing, Yanyu Ren, Shuntian Yao, Yuxiao Dong, and Jie Tang. Computerrl: Scaling end-to-end online reinforcement learning for computer use agents. *arXiv preprint arXiv:2508.14040*, 2025.
 - [19] Michael Luo, Sijun Tan, Roy Huang, Xiaoxiang Shi, Rachel Xin, Colin Cai, Ameen Patel, Alpay Ariyak, Qingyang Wu, Ce Zhang, et al. Deepcoder: A fully open-source 14b coder at o3-mini level, 2025.
 - [20] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Tianjun Zhang, Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. <https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2>, 2025. Notion Blog.
 - [21] Zhiyu Mei, Wei Fu, Kaiwei Li, Guangju Wang, Huanchen Zhang, and Yi Wu. Realhf: Optimized rlhf training for large language models through parameter reallocation. *arXiv preprint arXiv:2406.14088*, 2024.
 - [22] Zhiyu Mei, Wei Fu, Kaiwei Li, Guangju Wang, Huanchen Zhang, and Yi Wu. Real: Efficient rlhf training of large language models with parameter reallocation. In *Proceedings of the Eighth Conference on Machine Learning and Systems, MLSys 2025, Santa Clara, CA, USA, May 12-15, 2025*. mlsys.org, 2025.
 - [23] Erfan Miah and Eugene Belilovsky. Understanding and exploiting weight update sparsity for communication-efficient distributed rl. *arXiv preprint arXiv:2602.03839*, 2026.
 - [24] Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hosseini, Rishabh Agarwal, and Aaron Courville. Asynchronous rlhf: Faster and more efficient off-policy rl for language models. *arXiv preprint arXiv:2410.18252*, 2024.
 - [25] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
 - [26] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
 - [27] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
 - [28] Gerald Shen, Zhilin Wang, Olivier Delalleau, Jiaqi Zeng, Yi Dong, Daniel Egert, Shengyang Sun, Jimmy Zhang, Sahil Jain, Ali Taghibakhshi, et al. Nemo-aligner: Scalable toolkit for efficient model alignment. *arXiv preprint arXiv:2405.01481*, 2024.

- [29] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.
- [30] Yifan Sun, Jingyan Shen, Yibin Wang, Tianyu Chen, Zhendong Wang, Mingyuan Zhou, and Huan Zhang. Improving data efficiency for llm reinforcement fine-tuning through difficulty-targeted online data selection and rollout replay. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [31] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- [32] Shijie Wang, Pengfei Li, Yikun Fu, Kaifeng Liu, Fangyuan Li, Yang Liu, Xiaowei Sun, Zonglin Li, Siyao Zhao, Jian Zhao, et al. Marti-mars2: Scaling multi-agent self-search via reinforcement learning for code generation. *arXiv preprint arXiv:2602.07848*, 2026.
- [33] Yinjie Wang, Tianbao Xie, Ke Shen, Mengdi Wang, and Ling Yang. Rlanything: Forge environment, policy, and reward model in completely dynamic rl system. *arXiv preprint arXiv:2602.02488*, 2026.
- [34] Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. Swe-rl: Advancing llm reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*, 2025.
- [35] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First conference on language modeling*, 2024.
- [36] Yongji Wu, Xueshen Liu, Haizhong Zheng, Juncheng Gu, Beidi Chen, Z Morley Mao, Arvind Krishnamurthy, and Ion Stoica. Rlboost: Harvesting preemptible resources for cost-efficient reinforcement learning on llms. *arXiv preprint arXiv:2510.19225*, 2025.
- [37] Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. Less: selecting influential data for targeted instruction tuning. In *Proceedings of the 41st International Conference on Machine Learning*, pages 54104–54132, 2024.
- [38] Yixuan Even Xu, Yash Savani, Fei Fang, and Zico Kolter. Not all rollouts are useful: Down-sampling rollouts in llm reinforcement learning. *arXiv preprint arXiv:2504.13818*, 2025.
- [39] Ran Yan, Youhe Jiang, Tianyuan Wu, Jiaxuan Gao, Zhiyu Mei, Wei Fu, Haohui Mai, Wei Wang, Yi Wu, and Binhang Yuan. Areal-hex: Accommodating asynchronous rl training over heterogeneous gpus. *arXiv preprint arXiv:2511.00796*, 2025.
- [40] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.
- [41] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [42] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- [43] Yu Yue, Yufeng Yuan, Qiyang Yu, Xiaochen Zuo, Ruofei Zhu, Wenyuan Xu, Jiaze Chen, Chengyi Wang, TianTian Fan, Zhengyin Du, et al. Vapo: Efficient and reliable reinforcement learning for advanced reasoning tasks. *arXiv preprint arXiv:2504.05118*, 2025.
- [44] Hanchen Zhang, Xiao Liu, Bowen Lv, Xueqiao Sun, Bohao Jing, Iat Long Long, Zhenyu Hou, Zehan Qi, Hanyu Lai, Yifan Xu, Rui Lu, Hongning Wang, Jie Tang, and Yuxiao Dong. Agentrl: Scaling agentic reinforcement learning with a multi-turn, multi-task framework, 2025. URL <https://arxiv.org/abs/2510.04206>.
- [45] Yujie Zhao, Lanxiang Hu, Yang Wang, Minmin Hou, Hao Zhang, Ke Ding, and Jishen Zhao. Stronger-mas: Multi-agent reinforcement learning for collaborative llms. *arXiv preprint arXiv:2510.11062*, 2025.
- [46] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

- [47] Haizhong Zheng, Yang Zhou, Brian R Bartoldson, Bhavya Kailkhura, Fan Lai, Jiawei Zhao, and Beidi Chen. Act only when it pays: Efficient reinforcement learning for llm reasoning via selective rollouts. *arXiv preprint arXiv:2506.02177*, 2025.
- [48] Haizhong Zheng, Jiawei Zhao, and Beidi Chen. Prosperity before collapse: How far can off-policy rl reach with stale data on llms? In *ICLR*, 2026.
- [49] Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and Pengfei Liu. Deepresearcher: Scaling deep research via reinforcement learning in real-world environments. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 414–431, 2025.
- [50] Yinmin Zhong, Zili Zhang, Xiaoni Song, Hanpeng Hu, Chao Jin, Bingyang Wu, Nuo Chen, Yukun Chen, Yu Zhou, Changyi Wan, et al. Streamrl: Scalable, heterogeneous, and elastic rl for llms with disaggregated stream generation. *arXiv preprint arXiv:2504.15930*, 2025.
- [51] Yinmin Zhong, Zili Zhang, Bingyang Wu, Shengyu Liu, Yukun Chen, Changyi Wan, Hanpeng Hu, Lei Xia, Ranchen Ming, Yibo Zhu, et al. Optimizing {RLHF} training for large language models with stage fusion. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 489–503, 2025.
- [52] Zilin Zhu, Chengxing Xie, Xin Lv, and slime Contributors. slime: An llm post-training framework for rl scaling. <https://github.com/THUDM/slime>, 2025. GitHub repository.

Appendix

Limitations

AstraFlow focuses on system abstractions for agentic RL workloads rather than proposing a new RL optimization algorithm. Our evaluation covers representative math, coding, multi-policy, elastic rollout, heterogeneous deployment, and data-algorithm workloads, but does not exhaust all agentic RL settings, such as long-horizon web agents, robotics environments, or safety-critical interactive systems. The observed benefits may depend on workload characteristics including rollout latency, trainer throughput, network bandwidth, weight-transfer frequency, and asynchronous data availability. In addition, some experiments use controlled or simulated deployment settings, and AstraFlow assumes that rollout tasks, trajectory metadata, and reward signals can be represented through the dataflow layer. While AstraFlow makes multi-policy and data-centric RL workloads easier to compose, model quality, robustness, and safety still depend on the underlying data, rewards, policies, and evaluation protocol.

Social Impact

AstraFlow is an infrastructure system for reinforcement learning of agentic language models. Its positive impact is to make advanced RL experimentation more modular and resource-efficient by allowing researchers to reuse rollout services and trainers, compose data algorithms, and run flexible multi-policy or heterogeneous deployments without rewriting the full training pipeline. At the same time, more efficient RL infrastructure may accelerate stronger agentic models for coding, tool use, and autonomous decision-making, which could be misused without appropriate evaluation, monitoring, access control, or domain-specific safeguards. AstraFlow does not release a new foundation model or dataset, but users should still evaluate trained agents for reliability, security, privacy, and misuse risks before deployment.

A Experimental Settings

This appendix provides the full experimental setup deferred from Section 4. We organize the appendix per-experiment so that each subsection is self-contained and reproducible. Section A.1 fixes the shared bits that recur across experiments (model cards, evaluation suites, decoding, reward functions); Sections A.2–A.8 then specify, one per main-text experiment, the training data, algorithm hyperparameters, hardware and topology, baselines, and any experiment-specific knobs.

A.1 Common Setup

Evaluation protocol. Across *all* experiments in this appendix, we evaluate each benchmark by sampling 4 independent generations per question (temperature 0.6, $n_{\text{samples}}=1$) and reporting the mean per-question accuracy averaged over the benchmark. For every run we evaluate at a fixed cadence during training and report numbers from the checkpoint with the best benchmark-average accuracy. Per-experiment subsections below therefore omit this protocol and only note any deviations.

A.2 Multi-policy Math (Section 4.1, Table 2)

Models & Datasets. Both Solver and Verifier are initialized from Qwen3-8B [41] and trained as separate policies under the actor-and-verify workflow described in Section 4.1. We train on the DAPO RL math set [42] (Hugging Face: `aaabiao/dapo_filter`), filtering prompts to at most 2,000 tokens; following Dr. MAS [3], the trailing “Let’s think step by step ... `\boxed{\}`” suffix is stripped at load time so the multi-agent workflow owns response formatting. Verbatim prompts are listed in Appendix A.9.

Training. Both policies are trained with M2PO (m^2 -threshold 0.01), with group-level reward normalization, batch-level advantage normalization, and a fixed KL penalty coefficient of 10^{-3} (no KL controller). We use AdamW with a constant learning rate of $5e-6$ ($\beta_1=0.9$, $\beta_2=0.999$, weight decay 0.01), no warmup, and gradient clipping at 1.0. The training batch size is 256 with 8 rollouts per prompt and 4 PPO mini-batches per iteration;

rollout generation uses temperature 1.0 and $\text{max_new_tokens}=4,096$. We deliberately disable dynamic sampling in this run to align with the Dr. MAS [3] no-DS configuration that the Solver and Solver+Verifier (verl) baselines in Table 2 report against; this keeps the rollout regime matched across the three rows. Training runs for 1,200 iterations on a single 8×NVIDIA H100 (80 GB) node, with 4 GPUs serving the two SGLang RaaS instances (Solver and Verifier each at data-parallel size 2) and 4 GPUs split into two FSDP trainer groups of size 2 (one per policy). Trainer↔RaaS weight transfer runs in TCP delta mode with a full-sync interval of 10 iterations.

Baseline and time measurement. The Solver and Solver+Verifier (verl) accuracy rows in Table 2 are the corresponding numbers reported in Dr. MAS [3] (Qwen3-8B, GRPO over the same DAPO math set, no DS), which we adopt unchanged. The per-iteration training-time entries (212.64s for verl, 77.65s for AstraFlow), in contrast, are both measured on the same 8×H100 node used for the AstraFlow run: we re-run the verl-based Dr. MAS pipeline end-to-end on this hardware and time each iteration with eval steps excluded. For verl the iteration is rollout+train run sequentially under colocated synchronous execution; for AstraFlow the iteration is the trainer step time, with rollout overlap absorbed by the dataflow layer.

Evaluation. We evaluate on AIME24, AIME25, MATH500, and Minerva Math every 25 training iterations. Following the shared protocol in Section A.1, we sample 4 generations per question at temperature 0.6 and report $\text{pass@1}(\text{avg@4})$.

A.3 Multi-policy Code (Section 4.1, Table 3)

Models & Datasets. The three rows of Table 3 (single-policy Solver, Solver + Selector, Solver + Test-Case Generator) all initialize from Qwen3-8B [41] and use the workflows described in Section 4.1; multi-agent runs train each role as a separate policy. We train on the DeepCoder-Preview-Dataset [19] (Hugging Face: [agentica-org/DeepCoder-Preview-Dataset](https://huggingface.co/datasets/agentica-org/DeepCoder-Preview-Dataset), primeintellect subset, train split) and filter prompts to at most 6,000 tokens. Reward in all three runs is the binary outcome of executing the final attempt against the problem’s hidden test cases. Verbatim Solver, Selector, and Test-Case Generator prompts are listed in Appendix A.9.

Training. All three runs use the same M2PO settings as Section A.2, except: learning rate $3\text{e-}6$, 1,000 training iterations, dynamic sampling on (zero-advantage groups are filtered before training), and TCP *full*-sync weight transfer instead of delta. The training batch size is 256 with 8 rollouts per prompt; in the Solver + Selector run the Selector trainer uses a smaller batch size of 128, matching its lower per-iteration production rate. All runs use NVIDIA H200 (141 GB) GPUs. The Solver baseline runs on a single 8×H200 node, with 4 GPUs serving the policy (data-parallel size 4) and 4 GPUs as one FSDP trainer group of size 4. The two multi-agent runs each use two 8×H200 nodes (16 GPUs total): one node fully dedicated to RaaS hosting both policies, and the other node split into 4 RaaS GPUs and 4 FSDP trainer GPUs (two groups of size 2, one per policy). Rollout data-parallel sizes are 6 + 6 (Solver + Selector) and 9 + 3 (Solver + Test-Case Generator); the asymmetric v3 split reflects the Test-Case Generator’s lower per-rollout invocation rate.

Evaluation. We evaluate on LiveCodeBench v5 [15], LiveCodeBench v6, and the Codeforces split of DeepCoder-Preview-Dataset every 25 training iterations; multi-agent runs are scored end-to-end through the same workflow used in training. Following the shared protocol in Section A.1, we sample 4 generations per question at temperature 0.6 and report $\text{pass@1}(\text{avg@4})$.

A.4 Rollout Auto-scaling (Section 4.2.1, Table 4)

Models & Datasets. A single Qwen3-14B [41] policy is trained on the DeepScaler RL math set [20], filtering prompts to at most 2,000 tokens.

Training. We use M2PO with m^2 -threshold 0.002, group-level reward and batch-level advantage normalization, and a fixed KL penalty coefficient of 10^{-3} (no KL controller). We optimize with AdamW at a constant learning rate of $3\text{e-}6$ ($\beta_1=0.9$, $\beta_2=0.999$, weight decay 0.01), no warmup, and gradient clipping at 1.0. The training batch size is 256 with 8 rollouts per prompt and 4 PPO mini-batches per iteration; rollout generation uses temperature 1.0 with $\text{max_new_tokens}=18,000$ and SGLang context length 20,480; dynamic sampling is on (zero-advantage groups are filtered before training). Training runs for 1,200 iterations on two 8×NVIDIA H200 (141 GB) nodes (16 GPUs total). The trainer is fixed at 4 GPUs as a single FSDP group of size 4 on the main node; the remaining 4 GPUs on the main node and all 8 GPUs on the secondary node form the rollout pool. Each SGLang RaaS instance runs at tensor-parallel size 1 and is sized at data-parallel 1, 2, or 4 so that the

controller can compose pool sizes by adding or retiring instances. Trainer $\leftrightarrow$ RaaS weight transfer uses TCP full-sync mode.

Auto-scaling controller. We compare three rollout-pool configurations: a fixed pool of 6 GPUs, a fixed pool of 11 GPUs, and our auto-scaled pool, which the controller resizes between 6 and 11 GPUs. The dataflow layer exports the trainer waiting fraction w , recent production/consumption counts, and per-instance availability every $K=10$ trainer steps. We use the three-zone policy from Section 4.2.1 with lower and upper waiting-fraction thresholds $\tau_{\text{low}}=0.05$ and $\tau_{\text{high}}=0.10$ and a shrink-margin factor of $\rho=1.10$. The auto-scaling loop is executed by Claude Code as an agentic maintainer: at each report it reads the suggested G_{target} and issues the corresponding shell commands to launch or retire RaaS instances, preferentially removing unavailable or persistently low-throughput services on scale-down.

Balance report. The dataflow layer emits the balance report shown below every $K=10$ trainer versions. It is the sole signal the agentic maintainer consumes for scaling decisions. The report has three blocks: (i) a *window* block summarizing wall-clock time and the trainer waiting fraction $w = \sum_k \text{wait}_k / \sum_k \text{step}_k$ over the window; (ii) a *production* block aggregating produced/accepted/consumed tokens across all RaaS instances; (iii) a *scaling decision* block reporting the three-zone branch and the target GPU count G_{target} . A per-RaaS layout table follows so the maintainer can preferentially retire suspect or low-throughput instances on scale-down. *stale_skipped* is advisory and does not enter the scaling math.

Balance Report Template

```
--- Window (last <N> iterations) ---
wall_time_sec      : <wall_time>
eval_time_sec      : <eval_time>
training_time_sec   : <training_time>
avg_step_time_sec   : <avg_step_time>
avg_batch_wait_sec  : <avg_wait>
rollout_wait_fraction : <wait_fraction>

--- Production ---
total_raas_gpus     : <G>
produced            : <produced>
entered            : <accepted>
  accept_rate       : <accepted/produced>
consumed           : <consumed>
stale_skipped       : <stale>
  stale_rate        : <stale/accepted>
throughput_per_gpu   : <accepted/G>
produce_consume_ratio : <accepted/consumed>

--- Scaling decision ---
branch              : <scale_up | scale_down | hold>
G_target            : <G_target>
estimated_delta_gpus : <G_target - G, signed>
weight_transfer_active : <true | false>

--- RaaS Instance Layout (last <N> iterations) ---
uid      gpus  produced  accepted  accept_rate  throughput/gpu  status
<uid_1>   <g_1>   <p_1>     <a_1>     <ar_1>       <tpg_1>         healthy
<uid_2>   <g_2>   <p_2>     <a_2>     <ar_2>       <tpg_2>         suspect
...
---
Total: <M> instances, <sum_g> GPUs, <sum_p> produced, <sum_a> accepted
```

The three-zone decision rule the agent reads off the **branch** field is: scale up via $G_{\text{target}} = \lceil G / (1 - w) \rceil$ when $w > \tau_{\text{high}}=0.10$; scale down via $G_{\text{target}} = \min(G, \lceil G \cdot (\text{consumed/accepted}) \cdot \rho \rceil)$ with $\rho=1.10$ when $w < \tau_{\text{low}}=0.05$ and the window saw both production and consumption; otherwise hold.

Evaluation. We evaluate on AIME24, AIME25, AMC, MATH500, and Minerva Math every 50 training iterations. Following the shared protocol in Section A.1, we sample 4 generations per question at temperature 0.6 and report pass@1(avg@4).

A.5 Heterogeneous and Cross-region Training (Section 4.2.2)

Models & Datasets. A single Qwen3-14B [41] policy is trained on the DeepScaler RL math set [20], filtering prompts to at most 2,000 tokens.

Training. We use M2PO with m^2 -threshold 0.002, group-level reward and batch-level advantage normalization, and a fixed KL penalty coefficient of 10^{-3} (no KL controller). We optimize with AdamW at a constant learning rate of $3e-6$ ($\beta_1=0.9$, $\beta_2=0.999$, weight decay 0.01), no warmup, and gradient clipping at 1.0. The training batch size is 256 with 8 rollouts per prompt and 4 PPO mini-batches per iteration; rollout generation uses temperature 1.0 with $\text{max}_{\text{new_tokens}}=18,000$; dynamic sampling is on (zero-advantage groups are filtered before training). Training runs for 1,200 iterations. All training settings (model, dataset, M2PO hyperparameters, batch, iterations, decoding, dynamic sampling) are identical to the auto-scaling experiment in Section A.4; only the deployment topology (cross-region heterogeneous vs. homogeneous local) and the weight-transfer mode (delta vs. full sync) differ. The Fixed-11-GPU row in Table 4 (avg. accuracy 68.0) therefore serves as the homogeneous local baseline against which we compare the cross-region run’s 67.6 in Section 4.2.2.

Cross-region topology and heterogeneity. We simulate a cross-region heterogeneous deployment over three NVIDIA H200 (141 GB) nodes: a 4-GPU FSDP trainer (data-parallel size 4) on the local node, and three 4-GPU SGLang RaaS pools (each at tensor-parallel size 1, data-parallel size 4) — one co-located with the trainer and two on remote nodes. GPU heterogeneity is induced by per-GPU power caps: 700 W on the local pool, and 400 W and 250 W on the two remote pools, yielding rollout-throughput shares of roughly 100%, 60%, and 30% respectively (Figure 7b). The two remote links are shaped with `tc/netem` to 4 Gbit/s effective bandwidth and 300 ms round-trip latency; the local link is unshaped. Trainer $\leftrightarrow$ RaaS weight transfer uses TCP delta mode with a full-sync interval of 20 iterations; the periodic full syncs surface as the spikes in Figure 7c.

Evaluation. We evaluate on AIME24, AIME25, AMC, MATH500, and Minerva Math every 50 training iterations. Following the shared protocol in Section A.1, we sample 4 generations per question at temperature 0.6 and report `pass@1`(avg@4).

A.6 Delta-sparsity Measurement (Section 4.2.2, Figure 8)

Models & Datasets. We instrument six independent RL training runs that span two axes: (i) model scale, training Qwen3-1.7B / 8B / 14B [41] on the DeepScaler math set [20]; (ii) task, training Qwen2.5-7B-Instruct [40] on AlfWorld, WebShop, and search-based QA via the ASearcher workflow. All runs filter prompts to at most 2,000 tokens (math) or 4,096 tokens (agentic).

Training. Each underlying run is trained with M2PO with the same shared base settings as Section A.2 (group reward / batch advantage normalization, fixed KL penalty 10^{-3} , AdamW with $\beta_1=0.9$, $\beta_2=0.999$, weight decay 0.01, constant LR with no warmup, gradient clipping 1.0, 4 PPO mini-batches, dynamic sampling on, 8 rollouts per prompt at temperature 1.0); the per-run differences are summarized in Table 6. For the lr-ablation on Search, we additionally train the same workload at $\text{lr}=5e-6$ to study the effect of larger updates on sparsity. Hardware is NVIDIA H100 (80 GB), with a 4-GPU FSDP trainer and a 4-GPU SGLang RaaS pool per run.

Table 6 Per-run hyperparameters for the delta-sparsity measurement runs.

Workload	Model	lr	m^2 thresh.	Batch	Iters	$\text{max}_{\text{new_tok}}$
DeepScaler math	Qwen3-1.7B	$5e-6$	0.01	256	800	14,000
DeepScaler math	Qwen3-8B	$5e-6$	0.01	256	800	14,000
DeepScaler math	Qwen3-14B	$3e-6$	0.002	256	1,200	14,000
AlfWorld	Qwen2.5-7B-Instruct	$1e-6$	0.004	128	1,200	512
WebShop	Qwen2.5-7B-Instruct	$1e-6$	0.004	256	1,200	512
Search (ASearcher)	Qwen2.5-7B-Instruct	$1e-6$	0.004	256	1,000	1,024

Measurement. For each run we snapshot the trainer’s bf16 weights at every iteration and compute, for each consecutive pair of snapshots, the fraction of parameters that are bit-exactly equal to the previous iteration. Figure 8 reports this fraction averaged over the first 500 training iterations, one bar per run. Per-iteration sparsity curves over the full training horizon are in Appendix B.2.

A.7 Performance vs. AReaL (Section 4.2.3, Table 5)

Models & Datasets. We compare AstraFlow against AReaL [5] on two math jobs at different model scales: a single Qwen3-1.7B policy and a single Qwen3-8B policy, both trained on the DeepScaler RL math set [20] with prompts filtered to at most 2,000 tokens. Both frameworks use the same model initialization, training data, training budget, and algorithm hyperparameters; only the framework implementation differs.

Training. Both jobs train with M2PO using the same configuration: m^2 -threshold 0.01, group-level reward and batch-level advantage normalization, fixed KL penalty coefficient of 10^{-3} (no KL controller), AdamW with a constant learning rate of $5e-6$ ($\beta_1=0.9$, $\beta_2=0.999$, weight decay 0.01), no warmup, and gradient clipping at 1.0. The training batch size is 256 with 8 rollouts per prompt and 4 PPO mini-batches per iteration; rollout generation uses temperature 1.0 with $\max_{\text{new_tokens}} = 14,000$ and SGLang context length 16,384; dynamic sampling is on (zero-advantage groups are filtered before training). Each job runs for 800 iterations on a single $8 \times \text{NVIDIA H100}$ (80 GB) node, with 4 GPUs serving the policy (data-parallel size 4, tensor-parallel size 1) and 4 GPUs as one FSDP trainer group of size 4. Trainer $\leftrightarrow$ RaaS weight transfer uses TCP full-sync mode. The AReaL baseline reuses the exact same configuration — model initialization, training data, training budget, M2PO hyperparameters, generation settings, hardware layout, and weight-transfer mode — on the same single $8 \times \text{H100}$ node; only the framework implementation differs.

Evaluation. We evaluate on AIME24, AIME25, AMC, MATH500, and Minerva Math. Following the shared protocol in Section A.1, we sample 4 generations per question at temperature 0.6 and report pass@1(avg@4). We additionally report two efficiency numbers: median per-iteration training time, and total wall time normalized by the number of training tokens (Time/1M tok), both averaged after the warmup phase.

A.8 Data-algorithm Flexibility (Section 4.3, Figure 9)

Models & Datasets. A single Qwen3-8B [41] policy is trained on the DeepScaler RL math set [20], capped at the first 8,000 prompts and filtered to at most 2,000 tokens. We truncate the training set to 8,000 prompts (rather than the full split used in Sections A.4–A.7) because GRESO requires multiple passes over the same prompts to estimate per-prompt zero-variance probabilities and converge its bucket-level submit-probability schedule; capping the set ensures every variant runs through the prompt stream multiple times within the 800-iteration budget. This is also why the y -axis in Figure 9 sits a few points below the same-model accuracy reported on the full DeepScaler set in Section A.7. All four variants share this identical underlying prompt stream so that any difference in accuracy or rollout cost reflects the data algorithm rather than the data.

Training (shared). All variants train with M2PO using the same configuration: m^2 -threshold 0.01, group-level reward and batch-level advantage normalization, fixed KL penalty coefficient of 10^{-3} (no KL controller), AdamW with a constant learning rate of $5e-6$ ($\beta_1=0.9$, $\beta_2=0.999$, weight decay 0.01), no warmup, and gradient clipping at 1.0. The training batch size is 256 with 8 rollouts per prompt and 4 PPO mini-batches per iteration; rollout generation uses temperature 1.0 with $\max_{\text{new_tokens}} = 14,000$ and SGLang context length 16,384. Each variant runs for 800 iterations on a single $8 \times \text{NVIDIA H100}$ (80 GB) node, with 4 GPUs serving the policy (data-parallel size 4, tensor-parallel size 1) and 4 GPUs as one FSDP trainer group of size 4; weight transfer uses TCP full-sync mode.

Data-algorithm variants. Figure 9 compares three method families that compose dataflow-layer hooks at different intervention points: (i) *Vanilla* (1 point, $r=0$): the buffer applies the default `KeepAllFilter` and no replay or curator is enabled, so every prompt is submitted once per iteration and every generated trajectory is used. (ii) *DS + Replay* (4 points at replay ratios $r \in \{0.0, 0.3, 0.5, 0.7\}$): the post-rollout filter `filter_zero_adv` discards zero-advantage groups after generation [42], and the buffer reuses stored trajectories at training-batch serving time — a fraction r of each training batch is sampled from the replay pool (size 10,000, max staleness 8). At $r=0$ this collapses to dynamic sampling alone (the rightmost point on the curve, $\sim 720k$ rollouts). (iii) *DS + Replay + GRESO* (2 points at $r \in \{0.3, 0.5\}$): on top of `filter_zero_adv` and replay, GRESO [47] layers a pre-rollout curator that admits each prompt with a per-bucket submit probability adapting toward configured easy/hard zero-variance targets. We use the GRESO hyperparameters from [47]: initial easy/hard exploration probabilities 0.5/0.5, target zero-variance ratios $\alpha_{\text{easy}}=0.083$ and $\alpha_{\text{hard}}=0.167$, probability adjustment step $\Delta p=0.01$, per-prompt submit-probability floors 0.05 (easy) and 0.30 (hard), and a correctness threshold of 0.5 for easy/hard bucketing.

Evaluation. We evaluate on AIME24, AIME25, AMC, MATH500, and Minerva Math. Following the shared protocol in Section A.1, we sample 4 generations per question at temperature 0.6 and report the average pass@1 (avg@4) across the five benchmarks (the y -axis of Figure 9). The x -axis reports the cumulative number of generated rollouts over the 800 training iterations, exposing the rollout-cost / accuracy trade-off across data algorithms.

A.9 System Prompts for Single- and Multi-Policy Training

This subsection lists the verbatim prompts used in the single-policy and multi-policy collaborative-training experiments of Section 4.1. The math and code paths use different multi-agent workflow structures: math runs use a *symmetric solver-verifier* pair, where both agents share an environment context plus a teammate-output block and are conditioned on role-specific instructions; code runs use an asymmetric *solver + test-case generator* pipeline, where the solver first attempts the problem, the test-case generator synthesizes additional cases against which the solution is executed, and the solver is asked to retry with concrete failure feedback.

Math prompts. For single-policy math, we append a short reasoning suffix to the user’s question. For multi-policy math, the solver and verifier are instantiated from the same base model and trained jointly under the role-conditioned templates below; the placeholder {task_description} is filled with the raw problem text and {team_context} with the concatenated outputs of teammates accumulated so far in the rollout.

Single-Agent Math Suffix

Let’s think step by step. Please put your final answer within \boxed{ }.

Multi-Agent Solver/Actor Prompt (Math)

Task Introduction

You are a member of an expert multi-agent team tasked with solving the math problem. The team’s math problem is:

{task_description}

Your Teammates’ Outputs

{team_context}

Your Role

You are a “Solver Agent”. Your job is to carefully reason through the math problem step by step and derive the correct answer. When reasoning, consider your teammates’ outputs if available. Put the final answer in \boxed{ }.

Multi-Agent Verifier Prompt (Math)

Task Introduction

You are a member of an expert multi-agent team tasked with solving the math problem. The team’s math problem is:

{task_description}

Your Teammates’ Outputs

{team_context}

Your Role

You are a “Verifier Agent”. Your responsibility is to critically review the most recent solution provided by the “Solver Agent”. First, carefully examine each reasoning step, formula, and conclusion for accuracy, completeness, and logical consistency. Explain your analysis in detail. After completing your analysis, at the very end of your output, you MUST provide your final verdict within <verify> </verify> using exactly one of:

- (1) <verify>approve</verify> if all steps and the final answer are correct.
- (2) <verify>reject</verify> if you detect any issue.

Important: Do NOT output your verdict until you have finished your full analysis.

Code prompts. The Code Solver prompt below is used as the user message for both single-policy and multi-policy code training; it is the only prompt the single-policy solver ever sees. We evaluate two multi-agent code workflows that pair the solver with a second trained model in different ways: (i) *solver + test-case generator*, where the test-case generator synthesizes cases against which the candidate code is executed and, on failure, the solver retries with concrete failing-case feedback; (ii) *solver + selector*, where the solver produces two independent candidates from the same Code Solver prompt and a separate selector LLM is trained to pick the better one. The test-case generator and selector are each invoked with a system message that fixes their role and output discipline, paired with a user message supplying the problem statement and candidate(s); we list each system+user pair in a single box below, with role markers.

Code Solver Prompt (Single- and Multi-Agent)

Solve the following coding problem in Python 3.
Return only one final “python” code block containing the complete solution.
Question:
{question}

Multi-Agent Code Solver Retry Prompt

Solve the following coding problem in Python 3.
Return only one final “python” code block containing the complete solution.
Question:
{question}
Your previous solution failed on some test cases.
{feedback}
Now solve the problem and return the code.

Multi-Agent Code Test-Case Generator Prompt (Solver + Test-Case Generator workflow)

[System]
You are a testcase generator for a Python coding problem. Given the problem description (with built-in examples removed) and a candidate solution, produce exactly {generated_case_count} valid test cases that match the original interface. The cases must follow the problem statement, not the candidate solution. First think step by step about edge cases and potential bugs in the candidate inside an <analysis> </analysis> block. Then output exactly one “json fenced code block containing the final test cases, and nothing after it.

[User]
Problem
{problem_text}
Candidate Solution
{code_text}
Required Output Format
Return JSON with exactly this shape for {generated_case_count} cases:
{
 “fn_name”: “{fn_name}”,
 “inputs”: [<case1_args>, <case2_args>],
 “outputs”: [<case1_expected>, <case2_expected>]
}

Each entry in **inputs** must match the function-call format used by the original problem.

Multi-Agent Code Selector Prompt (Solver + Selector workflow)

[System]

You are a code selector. You will be shown a Python coding problem and two candidate solutions, Candidate A and Candidate B. Analyze both solutions for correctness: check the algorithm, edge cases, off-by-one errors, and whether each matches the problem specification. First think step by step inside an `<analysis>` `</analysis>` block. Then output exactly one of `<final>A</final>` or `<final>B</final>` on its own line and nothing after it.

[User]

Problem

{problem_text}

Candidate A

“python

{code_text_A}

“

Candidate B

“python

{code_text_B}

“

Decide which candidate is more likely to be correct (it is possible that neither is fully correct, or that both are; in those cases pick the one you judge more trustworthy). Reason inside an `<analysis>` `</analysis>` block, then emit exactly one of `<final>A</final>` or `<final>B</final>` on its own line and nothing after it.

A.10 System Prompts for Agentic Tasks

This subsection lists the prompts used by the agentic-task workloads referenced in Sections 4.2 and 4.3: AlfWorld and WebShop (both served via AgentBench task servers) and search-based QA (served via the ASearcher workflow). For AlfWorld and WebShop, the task server injects an instruction message followed by an in-context demonstration trajectory; below we list the system-level instructions verbatim and note where the few-shot / one-shot demos are appended.

AlfWorld. The task server first injects the task instruction below as a user message, then pre-loads a category-specific few-shot demonstration (one full successful trajectory per task type: **put**, **clean**, **heat**, **cool**, **examine**, **puttwo**) before the actual environment description (“Here is your task. `<observation>`”).

AlfWorld Task Instruction

Interact with a household to solve a task. Imagine you are an intelligent agent in a household environment and your target is to perform actions to complete the task goal. At the beginning of your interactions, you will be given the detailed description of the current environment and your goal to accomplish. For each of your turn, you will be given a list of actions which you can choose one to perform in this turn. You should choose from two actions: “THOUGHT” or “ACTION”. If you choose “THOUGHT”, you should first think about the current condition and plan for your future actions, and then output your action in this turn. Your output must strictly follow this format: “THOUGHT: your thoughts.

ACTION: your next action

”; If you choose “ACTION”, you should directly output the action in this turn. Your output must strictly follow this format: “ACTION: your next action

”. After your each turn, the environment will give you immediate feedback based on which you plan your next few steps. If the environment output “Nothing happened”, that means the previous action is invalid and you should try more options.

Reminder:

1. The action must be chosen from the given available actions. Any actions except provided available actions will be regarded as illegal.
2. Think when necessary, try to act directly more in the process.

WebShop. The task server injects the system-level prompt below, then pre-loads a one-shot demonstration of a complete shopping trajectory (search → click product → select size → buy) before the agent receives the actual instruction and observation.

WebShop Task Instruction

You are web shopping.
 I will give you instructions about what to do.
 You have to follow the instructions.
 Every round I will give you an observation and a list of available actions, you have to respond an action based on the state and instruction.
 You can use search action if search is available.
 You can click one of the buttons in clickables.
 An action should be of the following structure:
 search[keywords]
 click[value]
 If the action is not valid, perform nothing.
 Keywords in search are up to you, but the value in click must be a value in the list of available actions.
 Remember that your keywords in search should be carefully designed.
 Your response should use the following format:
 Thought:
 I think ...
 Action:
 click[something]

Search-based QA (ASearcher). The ASearcher workflow wraps the user question into a single rendered prompt that defines the search/answer protocol and pre-starts the assistant’s response with a `¡think¿` tag. Our experiments use the search-only variant (the agent has access to a single `¡search¿` tool); a richer search-and-access variant that additionally exposes an `¡access¿` tool for page retrieval is also implemented in the codebase but is not used in this paper’s evaluation.

Search-Only Prompt

A conversation between User and Assistant. The user asks a question, and the Assistant answers it. The Assistant analyzes the given question and information in the mind, retains important relevant information, calls a search engine to find necessary information, accesses web pages with certain urls, and provides the user with the answer. The Assistant conducts search by `<search> query </search>` and the top search results will be returned between `<information>` and `</information>`. The reasoning processes are enclosed within `<think> </think>`. Finally, the Assistant provides answer inside `<answer>` and `</answer>`, i.e. `<answer> answer here </answer>`. If there are multiple queries, ensure all answers are enclosed within `<answer> </answer>`, separated with comma. Note that when the Assistant finds the question is invalid, e.g. no answer could match all information in the question, the Assistant replies with ‘`<answer> the question is invalid. </answer>`’.

User:
 {question}.
 The language of your answer should align with the question. Assistant: `¡think¿`

B Additional Results

B.1 Accuracy of Agentic-Task Runs

The Qwen2.5-7B-Instruct training runs introduced for the delta-sparsity measurement (Section A.6) also produce trained policies on three agentic tasks; we report their accuracy here for completeness. For each run we follow the shared evaluation protocol of Section A.1 (4 generations per question at temperature 0.6, mean accuracy)

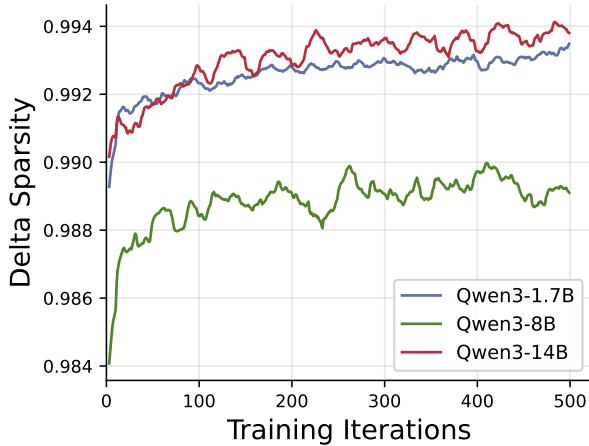
and report the checkpoint with the best benchmark-average score. Search (ASearcher) is evaluated on four open-domain QA suites and we additionally show a per-benchmark breakdown.

Table 7 Accuracy (pass@1 (avg@4), %) of the Qwen2.5-7B-Instruct M2PO runs from Section A.6, reported at the best-average checkpoint of each run.

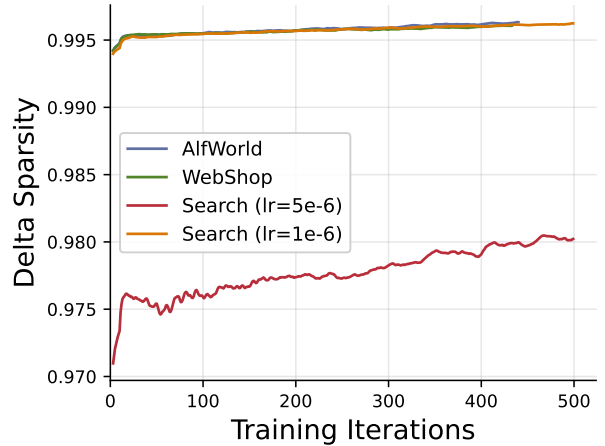
Task	Benchmark	Acc. (%)
AlfWorld	Valid	59.80
WebShop	Valid	93.63
Search (ASearcher)	Bamboogle	69.40
	HotpotQA	75.03
	PopQA	60.53
	TriviaQA	78.25
	Average	70.80

B.2 Per-Iteration Delta Sparsity

For completeness we also report the per-iteration sparsity curves underlying the averages in Figure 8. Each curve is averaged step-wise across overlapping sub-runs and then smoothed with a 15-iteration centered moving average; we restrict the x-axis to the first 500 training iterations.



(a) Math reasoning, Qwen3 [41] family.



(b) Tasks, Qwen2.5-7B-Instruct.

Figure 10 Per-iteration delta sparsity vs. training step, smoothed. (a) varies model scale on math (Qwen3-1.7B/8B/14B [41]); (b) varies task on Qwen2.5-7B-Instruct (AlfWorld, WebShop, Search at $lr = 1e-6$ and $5e-6$). The Search $lr=5e-6$ run is a clear outlier in (b); rerunning it at $lr=1e-6$ aligns it with AlfWorld and WebShop, confirming that learning-rate magnitude—not task type—drives the gap.